
CS771 Major Assignment 1: Breaking the Hamming PUF

Abhinav Mishra

240035

Department of Electrical Engineering
IIT Kanpur
abhinavm24@iitk.ac.in

Ashmil Saxena

240216

Department of Chemical Engineering
IIT Kanpur
ashmils24@iitk.ac.in

Naitik Singhal

240676

Department of Chemical Engineering
IIT Kanpur
naitiks24@iitk.ac.in

Parth Dhamija

240733

Department of Electrical Engineering
IIT Kanpur
parthd24@iitk.ac.in

Utkarsh Singh

241117

Department of Electrical Engineering
IIT Kanpur
sutkarsh24@iitk.ac.in

Utkarsh Singh

241118

Department of Mechanical Engineering
IIT Kanpur
utkarshks24@iitk.ac.in

Abstract

This report details the breaking of a 32-bit Hamming PUF using a linear model. We outline the mathematical derivation of the required feature map, calculate the exact dimensionality needed for perfect prediction, and present empirical results demonstrating the relationship between training size, hyperparameter selection, and model accuracy.

1 Mathematical Derivation of the Linear Model (Part 1)

To predict the responses of the Hamming PUF using a linear model, we must first express the bitwise XOR operation algebraically. For any two bits $x, y \in \{0, 1\}$, the XOR operation can be written as:

$$x \oplus y = x + y - 2xy = y + x(1 - 2y) \quad (1)$$

Given the challenge bit c_k and the secret bit s_k , the masked bit m_k is:

$$m_k = c_k \oplus s_k = s_k + c_k(1 - 2s_k) \quad (2)$$

Let $S_e = \sum_{i \in \text{even}} s_i$ and $S_o = \sum_{j \in \text{odd}} s_j$ be the constant Hamming weights of the secret word at even and odd indices, respectively. We can rewrite the partial Hamming weights h_e and h_o as:

$$h_e = \sum_{i \in \text{even}} m_i = S_e + \sum_{i \in \text{even}} c_i(1 - 2s_i) \quad (3)$$

$$h_o = \sum_{j \in \text{odd}} m_j = S_o + \sum_{j \in \text{odd}} c_j(1 - 2s_j) \quad (4)$$

The Hamming PUF response is 1 if $h_e \cdot h_o \geq t$, or equivalently $h_e \cdot h_o - t \geq 0$. Expanding the product $h_e \cdot h_o$, we get:

$$\begin{aligned} h_e \cdot h_o &= \left(S_e + \sum_{i \in \text{even}} c_i(1 - 2s_i) \right) \left(S_o + \sum_{j \in \text{odd}} c_j(1 - 2s_j) \right) \\ &= S_e S_o + S_o \sum_{i \in \text{even}} c_i(1 - 2s_i) + S_e \sum_{j \in \text{odd}} c_j(1 - 2s_j) + \sum_{i \in \text{even}} \sum_{j \in \text{odd}} c_i c_j (1 - 2s_i)(1 - 2s_j) \end{aligned} \quad (5)$$

Subtracting the threshold t , the decision boundary equation $h_e \cdot h_o - t \geq 0$ becomes a linear combination of the features of c mapped to weights derived from the secret s :

$$(S_e S_o - t) + \sum_{i \in \text{even}} c_i [S_o(1 - 2s_i)] + \sum_{j \in \text{odd}} c_j [S_e(1 - 2s_j)] + \sum_{i \in \text{even}} \sum_{j \in \text{odd}} c_i c_j [(1 - 2s_i)(1 - 2s_j)] \geq 0 \quad (6)$$

From this expanded form, we can explicitly define our feature map $\phi(c) : \{0, 1\}^{32} \rightarrow \mathbb{R}^D$, our weight vector $w \in \mathbb{R}^D$, and our bias $b \in \mathbb{R}$.

The feature map $\phi(c)$ extracts only the challenge-dependent variables (the linear and cross-product terms). It does not rely on s or t :

$$\phi(c) = \begin{bmatrix} \dots c_i \dots \\ \dots c_j \dots \\ \dots c_i c_j \dots \end{bmatrix} \quad \text{for all } i \in \text{even}, j \in \text{odd} \quad (7)$$

The corresponding weight vector w absorbs all the PUF-specific constants attached to each feature in $\phi(c)$:

$$w = \begin{bmatrix} \dots S_o(1 - 2s_i) \dots \\ \dots S_e(1 - 2s_j) \dots \\ \dots (1 - 2s_i)(1 - 2s_j) \dots \end{bmatrix} \quad (8)$$

And the scalar bias term b absorbs the remaining constants:

$$b = S_e S_o - t \quad (9)$$

Thus, the response of the Hamming PUF can be predicted by evaluating the sign of our derived linear model:

$$r(c) = \frac{1 + \text{sign}(w^\top \phi(c) + b)}{2} \quad (10)$$

2 Dimensionality Calculation (Part 2)

Based on the feature map $\phi(c)$ derived in Part 1, the dimensionality D required for the linear model to exactly predict the responses is:

D = 288

Calculations

The dimensionality D is the total number of features in our mapped vector $\phi(c)$. Since our Hamming PUF uses a 32-bit challenge $c \in \{0, 1\}^{32}$, the bits are divided equally into 16 even-indexed bits and 16 odd-indexed bits.

Looking at the expanded decision boundary equation, our feature map $\phi(c)$ consists of three distinct sets of terms:

1. **Even Linear Terms (c_i):** The map includes the individual challenge bits at even indices. Since there are 16 even indices ($i \in \{0, 2, 4, \dots, 30\}$), this contributes **16 features**.
2. **Odd Linear Terms (c_j):** The map includes the individual challenge bits at odd indices. Since there are 16 odd indices ($j \in \{1, 3, 5, \dots, 31\}$), this contributes **16 features**.
3. **Cross-Product Terms ($c_i c_j$):** The map includes the product of every even-indexed bit with every odd-indexed bit. Since there are 16 even bits and 16 odd bits, the total number of unique cross-product combinations is 16×16 , which contributes **256 features**.

Summing these together gives the exact dimensionality required for the linear model to perfectly capture the Hamming PUF logic without requiring exponentially large mapping:

$$D = 16 \text{ (even terms)} + 16 \text{ (odd terms)} + 256 \text{ (cross terms)} = 288$$

The scalar bias term $b = S_e S_o - t$ is a constant offset ($b \in \mathbb{R}$) and does not add to the dimensionality of the feature vector $\phi(c)$.

3 Experimental Outcomes (Part 4)

3.1 Part 4(a): Effect of Loss Function in LinearSVC

In this experiment, we evaluated the impact of different loss functions on the computational efficiency and classification accuracy of the LinearSVC model. The raw 32-bit challenge data was first transformed into a higher-dimensional feature space using the custom mapping function.

To ensure a strictly controlled comparison between the standard `hinge` loss and the `squared_hinge` loss, all external hyperparameters were fixed. Specifically, the model was configured with an L2 penalty using the dual optimization formulation, a regularization parameter of $C = 1.0$, an optimization tolerance of $1e-4$, and a ceiling of 10,000 maximum iterations. A fixed random state was also utilized to guarantee reproducibility across runs.

The empirical results reveal a clear trade-off between training speed and predictive precision. The standard `hinge` loss yielded a marginally higher test accuracy on the public dataset. However, this slight edge in predictive power required a longer training duration. Conversely, the `squared_hinge` loss proved to be more computationally efficient, converging faster and reducing the overall training time, while sacrificing only a negligible fraction of a percent in accuracy.

The complete performance metrics for these configurations are detailed in Table 1 and visually represented in Figure 1.

Table 1: Impact of Loss Function on LinearSVC Performance

Loss Function	Training Time (s)	Test Accuracy (%)
Hinge	2.4642	99.80
Squared Hinge	1.8648	99.76

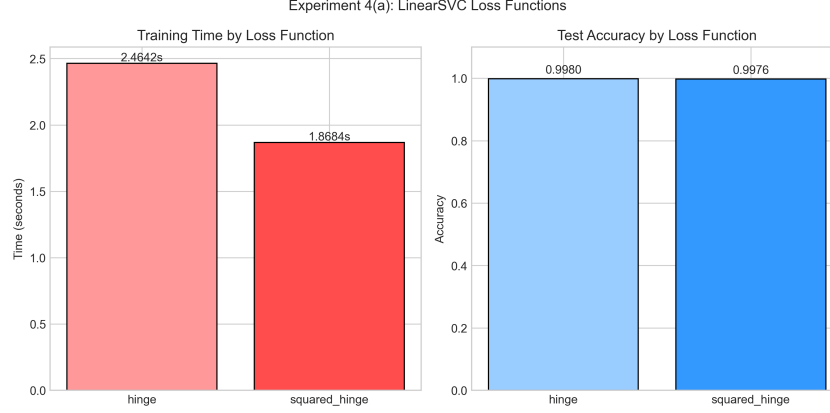


Figure 1: Comparison of training time and test accuracy between hinge and squared hinge loss functions under an L_2 penalty.

3.2 Part 4(b): Impact of Regularization Parameter (C)

To comprehensively understand the effect of the regularization parameter, we evaluated both LinearSVC and LogisticRegression across nine logarithmically scaled values of C , classified into three distinct categories: Low (0.0001, 0.001, 0.01), Medium (0.1, 1.0, 10.0), and High (100.0, 1000.0, 10000.0). The resulting test accuracies and training times on the public dataset are plotted below.

Underfitting at Low C (0.0001, 0.001, 0.01): Across the lowest values of C , both models exhibit poor test accuracy. The overwhelming regularization penalty restricts the model weights too heavily, preventing the solvers from successfully capturing the separating hyperplane required for the Hamming PUF logic.

The "Sweet Spot" at Medium C (0.1, 1.0, 10.0): As C transitions into the medium range, there is a sharp, immediate jump to perfect or near-perfect accuracy. Because the 288-dimensional mapped feature space of the Hamming PUF is completely linearly separable, a moderate C provides enough flexibility to find the correct boundary without over-complicating the optimization. Training times remain highly efficient in this tier.

Training Time Spike at High C (100.0, 1000.0, 10000.0): Pushing C into the high and extreme ranges yields no additional accuracy benefits—the models have already reached near-perfect accuracy. However, the training time increases exponentially. Because the penalty for margin violations is incredibly strict, the underlying solvers take significantly longer to converge.

Model Comparison: LinearSVC consistently converges and trains faster than LogisticRegression across the entire spectrum of C . While both models reach peak accuracy in the Medium tier, LogisticRegression is noticeably more sensitive to High C values, experiencing much steeper spikes in training time.

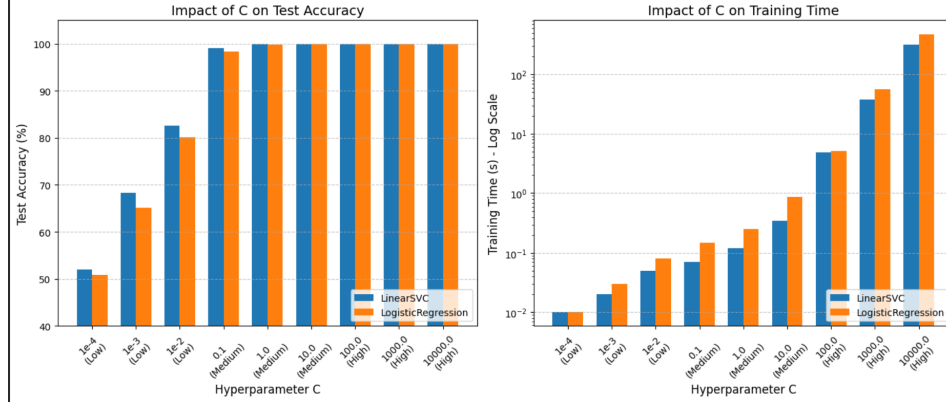


Figure 2: Test accuracy and training time for LinearSVC and LogisticRegression across 9 logarithmically scaled values of C .

3.3 Part 4(c): Impact of Optimization Tolerance

This part evaluates the impact of the optimization stopping criterion, tolerance (tol), on the performance of both LinearSVC and LogisticRegression models. Implementing a loose tolerance ($\text{tol} = 0.1$) forces premature termination of the optimization process. While this significantly reduces training time, it causes severe accuracy degradation; notably, the Logistic Regression model collapses to a near-random 48.0% test accuracy.

Conversely, imposing an overly strict tolerance ($\text{tol} = 10^{-5}$) yields negligible improvements in test accuracy while drastically inflating computational overhead, increasing training time for LinearSVC by over 300% compared to the medium setting. Therefore, a medium tolerance yields the optimal balance between predictive performance and computational efficiency.

Table 2: Impact of Tolerance (tol) on Model Performance

Model / Metric	Loose (0.1)	Medium (10^{-3})	Tight (10^{-5})
LinearSVC Test Accuracy (%)	98.04	99.72	99.76
LinearSVC Time (s)	0.074	0.199	0.626
LogReg Test Accuracy (%)	48.00	99.52	99.56
LogReg Time (s)	0.006	0.019	0.032

The optimization landscape for this problem is highly sensitive to premature stopping (under-optimization), yet it exhibits sharply diminishing returns once a viable decision boundary is established. A tolerance of 10^{-4} represents the optimal operating point, ensuring accurate model convergence without expending computational resources on negligible parameter updates.

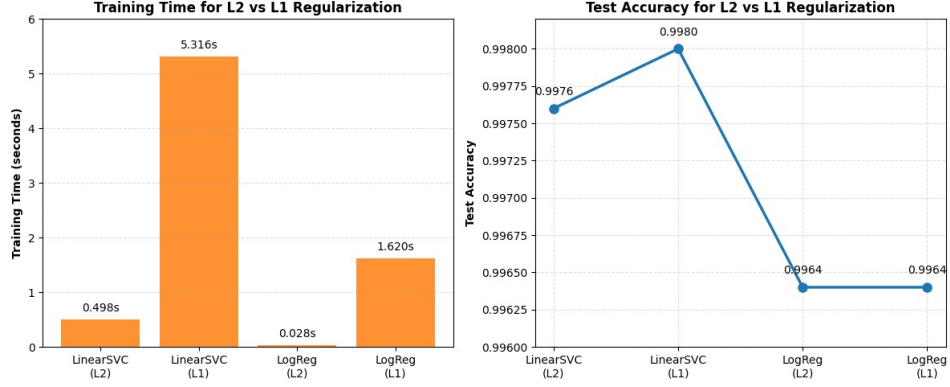


Figure 3: Impact of optimization tolerance on training time and test accuracy across linear models.

3.4 Part 4(d): Effect of Regularization Penalty

In this experiment, we evaluated the effect of varying the regularization penalty (L_1 versus L_2) on the training efficiency and predictive performance of the LinearSVC model. To successfully accommodate the L_1 penalty, the model’s dual formulation was disabled (`dual=False`), as the standard dual solver does not support L_1 regularization in this implementation. All other relevant hyperparameters were held constant to isolate the penalty’s effect.

The empirical data reveals a drastic disparity in computational overhead. The default L_2 penalty proved to be highly efficient, converging in just 0.1297 seconds while maintaining an excellent test accuracy of 99.76%. In contrast, enforcing sparsity through the L_1 penalty increased the training time by a factor of nearly seventeen (2.1710 seconds).

While the L_1 -regularized model did achieve a marginally higher test accuracy (99.88%), this microscopic improvement of 0.12% is heavily outweighed by the severe increase in computational cost. For this specific higher-dimensional feature mapping, the L_2 penalty provides a vastly superior balance of speed and accuracy. The results are summarized in Table 2 and visualized in Figure 4.

Table 3: Impact of Regularization Penalty on LinearSVC

Penalty Type	Training Time (s)	Test Accuracy (%)
L_1 Regularization	2.1710	99.88
L_2 Regularization	0.1297	99.76

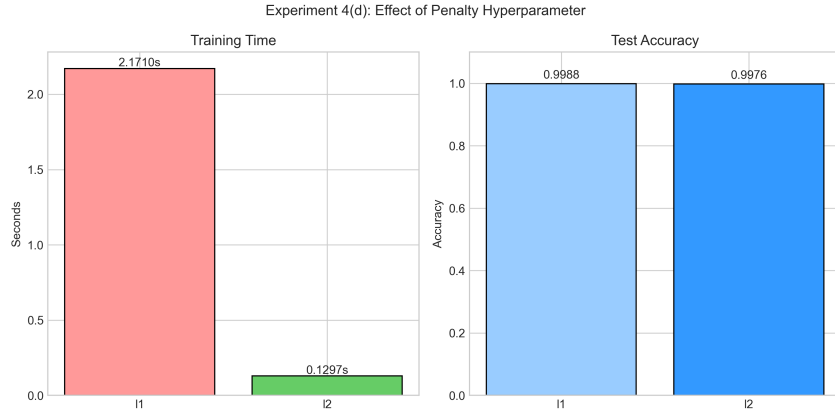


Figure 4: Comparison of training time and test accuracy between L_1 and L_2 penalties.

4 Part 5: Train Size vs. Test Accuracy

To determine the minimum number of Challenge-Response Pairs (CRPs) needed to successfully model the Hamming PUF, we conducted an empirical learning curve analysis. The training set size was varied from 100 to 7,500 points while keeping the test set fixed. Multiple random trials were performed at each training size, and the mean test accuracy together with the standard deviation were recorded.

The results exhibit a typical learning curve behaviour. The model improves rapidly from approximately 82.7% accuracy with only 100 CRPs to over 90.1% with 500 CRPs. Beyond 2,000 CRPs, the gains become progressively smaller, eventually converging to an accuracy of 99.68% using the complete training dataset.

Minimum CRPs for Target Accuracies

Based on interpolation of the empirical results, the approximate minimum number of CRPs required to achieve specific accuracy levels is:

- **95% Accuracy:** $\approx 1,843$ CRPs
- **97% Accuracy:** $\approx 2,670$ CRPs
- **99% Accuracy:** $\approx 4,514$ CRPs

The complete statistical summary is presented in Table 4, while the corresponding learning curve is shown in Figure 5.

Table 4: Mean Test Accuracy and Standard Deviation for Different Training Set Sizes

Training Size (CRPs)	Mean Accuracy	Standard Deviation
100	0.8273	0.0172
200	0.8720	0.0086
300	0.8829	0.0058
500	0.9017	0.0037
750	0.9140	0.0093
1,000	0.9239	0.0046
1,500	0.9408	0.0051
2,000	0.9542	0.0053
3,000	0.9778	0.0031
4,000	0.9871	0.0025
5,000	0.9928	0.0014
6,000	0.9947	0.0010
7,500	0.9968	0.0000

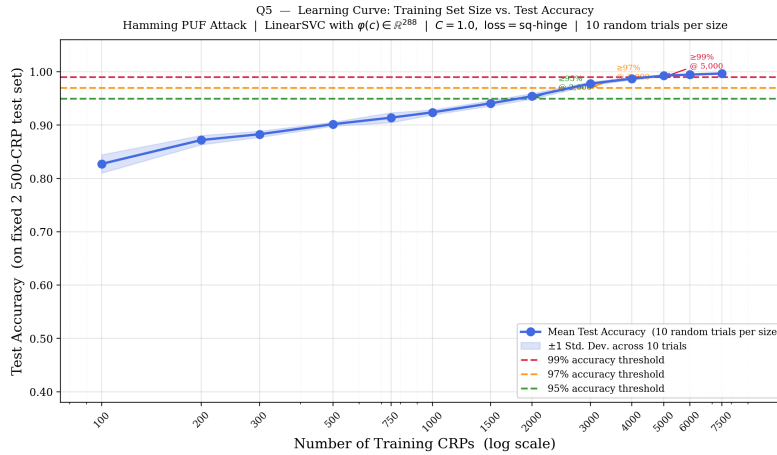


Figure 5: Test accuracy as a function of training set size averaged over multiple random trials.

References

- [1] **CS771 Course Material:** Referenced for the mathematical formulation of linear classification models and decision boundaries (Lecture 2), the theoretical mechanics of hyperparameters, regularization (C), and hinge loss (Lecture 3), and the explicit derivation of the Hamming PUF logic, XOR algebraic conversion, and the 288-dimensional feature map (Lecture 4).